

Exercise 7-1: Prepare the Forest Fires data

Did you use a language model?

A : No, I did not find any use for a language model for this assignment except for writing comments.

Read the data

```
In [1]: # Import the pandas library
import pandas as pd
```

```
In [2]: # Load a DataFrame from a pickle file named 'fires_by_month.pkl'. This file contains
fires_by_month = pd.read_pickle('fires_by_month.pkl')
```

```
In [3]: # Display the first 5 rows of the DataFrame to get an overview of the data. Use the
fires_by_month.head()
```

```
Out[3]:
```

	state	fire_year	fire_month	acres_burned	days_burning	fire_count
0	AK	1992	5	4202.0	135.0	14
1	AK	1992	6	86401.0	417.0	23
2	AK	1992	7	48516.7	500.0	26
3	AK	1992	8	3305.0	92.0	4
4	AK	1992	9	20.0	1.0	1

Add and modify columns

```
In [4]: # Add a new column 'mean_acres_per_day' to the DataFrame, calculating the average
fires_by_month['mean_acres_per_day'] = fires_by_month.acres_burned / fires_by_month.days_burning
```

```
In [5]: # Display the first 3 rows of the DataFrame to check the newly added 'mean_acres_per_day'
fires_by_month.head(3)
```

```
Out[5]:
```

	state	fire_year	fire_month	acres_burned	days_burning	fire_count	mean_acres_per_day
0	AK	1992	5	4202.0	135.0	14	31.1259
1	AK	1992	6	86401.0	417.0	23	207.1966
2	AK	1992	7	48516.7	500.0	26	97.0362

```
In [6]: # Add another new column 'mean_acres_per_day_lambda' using the apply method with
fires_by_month['mean_acres_per_day_lambda'] = fires_by_month.apply(
    lambda x: 0 if x.days_burning <= 0 else x.acres_burned / x.days_burning, axis=1
```

```
In [7]: # Display the first 3 rows of the DataFrame again to check the newly added 'mean_fires_by_month'.head(3)
```

```
Out[7]:
```

	state	fire_year	fire_month	acres_burned	days_burning	fire_count	mean_acres_per
0	AK	1992	5	4202.0	135.0	14	31.125
1	AK	1992	6	86401.0	417.0	23	207.196643
2	AK	1992	7	48516.7	500.0	26	97.033400

```
In [8]: # Define a function 'convert_month' that takes a row of the DataFrame as input.
# This function maps numeric month values (1-12) to their corresponding month abbreviations
def convert_month(row):
    month_dict = {1:'Jan', 2:'Feb', 3:'Mar', 4:'Apr',
                  5:'May', 6:'Jun', 7:'Jul', 8:'Aug',
                  9:'Sep', 10:'Oct', 11:'Nov', 12:'Dec'}
    # Access the 'fire_month' column of the given row, use it to find the corresponding month abbreviation
    return month_dict[row.fire_month]
```

```
In [9]: # Use the 'apply' method with a lambda function to apply the 'convert_month' function to the 'fire_month' column
# This effectively converts each numeric month value in the 'fire_month' column to its corresponding month abbreviation
# The result is then assigned back to the 'fire_month' column, updating it with the new month abbreviations
fires_by_month['fire_month'] = fires_by_month.apply(lambda x: convert_month(x), axis=1)

# Display the first 3 rows of the DataFrame to check the updated 'fire_month' column
fires_by_month.head(3)
```

```
Out[9]:
```

	state	fire_year	fire_month	acres_burned	days_burning	fire_count	mean_acres_per
0	AK	1992	May	4202.0	135.0	14	31.125
1	AK	1992	Jun	86401.0	417.0	23	207.196643
2	AK	1992	Jul	48516.7	500.0	26	97.033400

Work with indexes

```
In [10]: # Set the DataFrame index to a MultiIndex by specifying a list of columns ('state', 'fire_year', 'fire_month')
# This organizes the DataFrame by these three levels, allowing for more sophisticated indexing and data manipulation
fires_by_month.set_index(['state', 'fire_year', 'fire_month'], inplace=True)

# Display the first 3 rows of the DataFrame to observe the new MultiIndex structure
fires_by_month.head(3)
```

```
Out[10]:
```

	state	fire_year	fire_month	acres_burned	days_burning	fire_count	mean_acres_per_day
	AK	1992	May	4202.0	135.0	14	31.125926
			Jun	86401.0	417.0	23	207.196643
			Jul	48516.7	500.0	26	97.033400

```
In [11]: # Pivot the DataFrame to a wider format using the 'unstack' method, which moves
# This can be useful for comparing values across months more easily.
fires_by_month_wide = fires_by_month.unstack(level='fire_month')

# Display the first 3 rows of the transformed (wider) DataFrame to inspect the n
fires_by_month_wide.head(3)
```

```
Out[11]:
```

												acres_burned
	fire_month	Apr	Aug	Dec	Feb	Jan	Jul	Jun	Mar	May	Nov	
state	fire_year											
AK	1992	NaN	3305.0	NaN	NaN	NaN	48516.7	86401.0	NaN	4202.0	NaN	135.0
	1993	113.0	1132.0	NaN	NaN	NaN	483493.3	197740.2	NaN	4152.0	NaN	417.0
	1994	NaN	24139.0	NaN	NaN	NaN	50510.0	184428.0	200.0	1127.7	NaN	1.0

3 rows × 60 columns

```
In [12]: # Reset the index of the original DataFrame, moving the index levels back to col
# The resulting DataFrame is useful for operations that require a simple, non-hi
fires_by_month = fires_by_month.reset_index()

# Display the first 5 rows of the reset DataFrame to confirm that the index has
fires_by_month.head()
```

```
Out[12]:
```

	state	fire_year	fire_month	acres_burned	days_burning	fire_count	mean_acres_per_fire
0	AK	1992	May	4202.0	135.0	14	31.125
1	AK	1992	Jun	86401.0	417.0	23	207.196
2	AK	1992	Jul	48516.7	500.0	26	97.033
3	AK	1992	Aug	3305.0	92.0	4	35.925
4	AK	1992	Sep	20.0	1.0	1	20.000

Combine data

```
In [13]: # create new fire data
new_fire = pd.DataFrame(data=[['CA', 2021, 'June', 1000, 100, 1, 10, 10]], columns=fire
new_fire.head()
```

```
Out[13]:
```

	state	fire_year	fire_month	acres_burned	days_burning	fire_count	mean_acres_per_fire
0	CA	2021	June	1000	100	1	10.000

```
In [14]: # Create a new DataFrame 'new_fire' with a single row of data, representing a ne
new_fire = pd.DataFrame(data=[['CA', 2021, 'June', 1000, 100, 1, 10, 10]], columns=fire

# Display the 'new_fire' DataFrame to verify its content.
new_fire.head()
```

Out[14]:

	state	fire_year	fire_month	acres_burned	days_burning	fire_count	mean_acres_per_
0	CA	2021	June	1000	100	1	

In [15]:

```
# Concatenate the 'new_fire' DataFrame with the existing 'fires_by_month' DataFrame
fires_by_month = pd.concat([fires_by_month, new_fire])

# Display the last few rows of the updated 'fires_by_month' DataFrame to confirm
fires_by_month.tail()
```

Out[15]:

	state	fire_year	fire_month	acres_burned	days_burning	fire_count	mean_acres_
9296	WY	2015	Sep	5977.0	126.0	25	47
9297	WY	2015	Oct	10337.8	15.0	6	689
9298	WY	2015	Nov	509.3	3.0	4	169
9299	WY	2015	Dec	72.0	0.0	2	
0	CA	2021	June	1000.0	100.0	1	10

In [16]:

```
# Reset the index of the 'fires_by_month' DataFrame, dropping the old index and
fires_by_month.reset_index(drop=True, inplace=True)
```

In [17]:

```
# Display the last few rows of the 'fires_by_month' DataFrame again to confirm t
fires_by_month.tail()
```

Out[17]:

	state	fire_year	fire_month	acres_burned	days_burning	fire_count	mean_acres_
9296	WY	2015	Sep	5977.0	126.0	25	47
9297	WY	2015	Oct	10337.8	15.0	6	689
9298	WY	2015	Nov	509.3	3.0	4	169
9299	WY	2015	Dec	72.0	0.0	2	
9300	CA	2021	June	1000.0	100.0	1	10

Fix the SettingWithCopyWarning

In [18]:

```
# Filter the 'fires_by_month' DataFrame for records where the state is 'AK' (Ala
fires_ak = fires_by_month.query('state == "AK"').copy()

# Round the 'mean_acres_per_day' column in the 'fires_ak' DataFrame to the neare
fires_ak.mean_acres_per_day = fires_ak.mean_acres_per_day.round()

# Display the first few rows of the 'fires_ak' DataFrame to check the rounding o
fires_ak.head()
```

Out[18]:

	state	fire_year	fire_month	acres_burned	days_burning	fire_count	mean_acres_per_
0	AK	1992	May	4202.0	135.0	14	
1	AK	1992	Jun	86401.0	417.0	23	2
2	AK	1992	Jul	48516.7	500.0	26	
3	AK	1992	Aug	3305.0	92.0	4	
4	AK	1992	Sep	20.0	1.0	1	

In [19]:

```
# Finally, display the first few rows of the 'fires_by_month' DataFrame to confi
fires_by_month.head()
```

Out[19]:

	state	fire_year	fire_month	acres_burned	days_burning	fire_count	mean_acres_per_
0	AK	1992	May	4202.0	135.0	14	31.125
1	AK	1992	Jun	86401.0	417.0	23	207.196
2	AK	1992	Jul	48516.7	500.0	26	97.033
3	AK	1992	Aug	3305.0	92.0	4	35.923
4	AK	1992	Sep	20.0	1.0	1	20.000

In []: